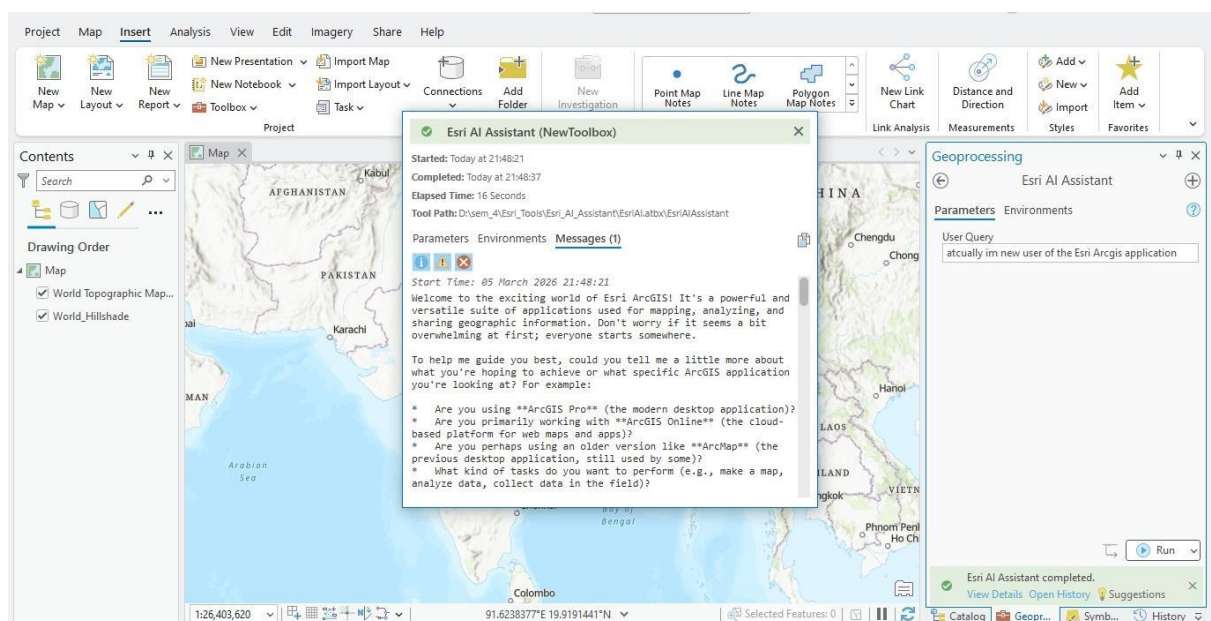
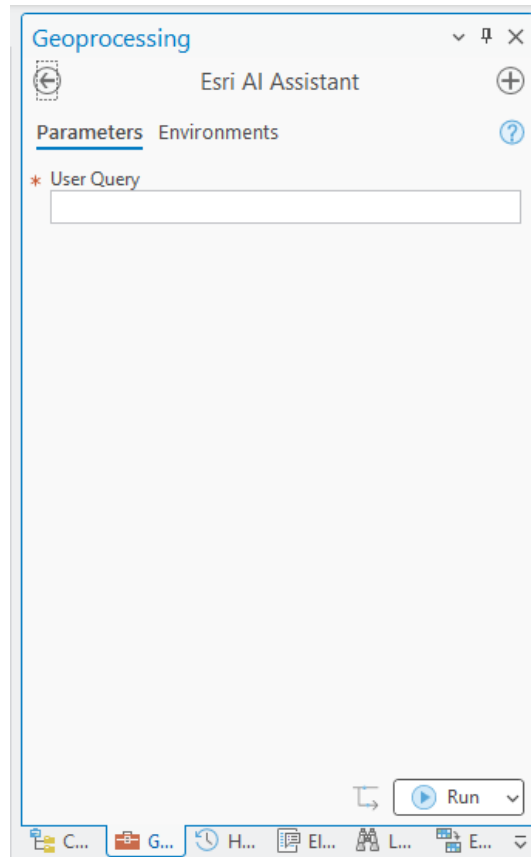


ESRI AI Agent – Code Documentation

1. Overview

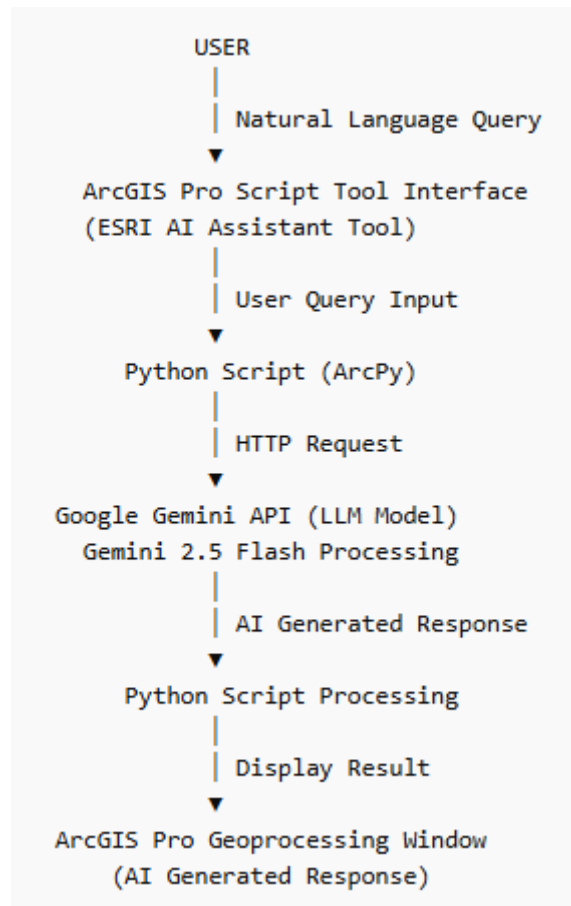


The **ESRI AI Agent** is a Python-based geoprocessing tool developed for **ArcGIS Pro** that integrates a **Large Language Model (LLM)** using the Google Gemini API. The tool enables users to interact with GIS software through **natural language queries**, allowing them to

obtain explanations, workflows, and geospatial guidance directly inside the ArcGIS environment.

The script captures a user query from the ArcGIS tool interface, sends it to the Gemini API, and displays the generated response in the ArcGIS geoprocessing message window.

2. System Architecture



3. Libraries Used

arcpy

ArcPy is the Python library provided by **ArcGIS Pro** for geospatial automation and geoprocessing.

Used for:

- Reading user parameters
- Displaying messages in ArcGIS
- Integrating the script with ArcGIS tools

requests

The **requests** library is used to send HTTP requests to the Gemini API.

Used for:

- Connecting to external APIs
- Sending user prompts
- Receiving AI-generated responses

4. Script Explanation (Step-by-Step)

Step 1 – Import Required Libraries

```
import arcpy
import requests
```

This section imports the required Python modules.

- **arcpy** enables communication with ArcGIS Pro.
- **requests** allows the script to interact with the Gemini API.

Step 2 – Capture User Input

```
user_prompt = arcpy.GetParameterAsText(0)
```

The `GetParameterAsText()` function retrieves user input from the ArcGIS geoprocessing tool interface.

Parameter index **0** corresponds to:

```
User Query
```

Example input from user:

```
What is NDVI in remote sensing?
```

Step 3 – Define API Key

```
API_KEY = "key"
```

This variable stores the **Google Gemini API key** required to authenticate requests.

Important

For security reasons, API keys should ideally be stored in:

- environment variables
- configuration files

instead of directly inside the script.

Step 4 – Define API Endpoint

```
url = f"https://generativelanguage.googleapis.com/v1/models/gemini-2.5-
flash:generateContent?key={API_KEY}"
```

This defines the **Gemini API endpoint** used to generate responses.

Model used:

Gemini 2.5 Flash

This model is optimized for:

- fast responses
- conversational tasks
- lightweight applications

Step 5 – Define Request Headers

```
headers = {
    "Content-Type": "application/json"
}
```

The header specifies that the request body is sent in **JSON format**, which is required by the API.

Step 6 – Prepare Request Data

```
data = {
    "contents": [
        {
            "parts": [
                {"text": user_prompt}
            ]
        }
    ]
}
```

This section constructs the **JSON payload** sent to the Gemini API.

Structure explanation:

```
contents
├── parts
│   └── text (user prompt)
```

Example request body:

```
{
  "contents": [
    {
      "parts": [
        {"text": "Explain NDVI in agriculture"}
      ]
    }
  ]
}
```

Step 7 – Send API Request

```
response = requests.post(url, headers=headers, json=data)
```

This line sends a **POST request** to the Gemini API.

Components:

Component	Description
url	API endpoint
headers	HTTP request header
json=data	JSON payload containing user query

The API processes the prompt and returns an AI-generated response.

Step 8 – Check API Response

```
if response.status_code == 200:
```

The script verifies whether the request was successful.

Status code **200** indicates:

```
Successful API request
```

If successful, the script proceeds to extract the generated text.

Step 9 – Extract AI Response

```
result = response.json()
output_text = result["candidates"][0]["content"]["parts"][0]["text"]
```

The API returns a structured JSON response.

The script navigates through the response to extract the generated text.

Structure of Gemini response:

```

candidates
├── content
│   └── parts
│       └── text

```

Example response:

```

{
  "candidates": [
    {
      "content": {
        "parts": [
          { "text": "NDVI is a vegetation index used to measure plant health..." }
        ]
      }
    }
  ]
}

```

Step 10 – Display Result in ArcGIS

```
arcpy.AddMessage(output_text)
```

The AI-generated text is displayed in the **ArcGIS geoprocessing message window**.

Example output in ArcGIS:

```
NDVI (Normalized Difference Vegetation Index) is used to measure vegetation health using NIR and Red bands.
```

Step 11 – Error Handling

```

else:
    arcpy.AddError("Error from Gemini API:")
    arcpy.AddError(str(response.json()))

```

If the API request fails, the script:

1. Displays an error message in ArcGIS
2. Prints the API error response for debugging

This helps users identify issues such as:

- invalid API key
- network errors
- incorrect API request

5. Example Workflow

Step 1

User opens the **ESRI AI Assistant tool**.

Step 2

User enters a question:

How to calculate NDVI in ArcGIS Pro?

Step 3

The script sends the prompt to the Gemini API.

Step 4

Gemini processes the request.

Step 5

The response is displayed inside ArcGIS.

6. Advantages of the Tool

The ESRI AI Agent provides several benefits:

- Enables **natural language interaction with GIS**
- Provides **instant geospatial explanations**
- Helps generate **GIS workflows and ArcPy scripts**
- Reduces the learning curve for new GIS users
- Integrates AI directly inside ArcGIS Pro

7. Limitations

Current limitations of the system include:

- Requires internet connectivity
- Dependent on external API availability
- Cannot directly execute generated GIS workflows

8. Future Enhancements

Possible improvements include:

- voice-based GIS queries
- map screenshot analysis
- automatic ArcPy script execution
- GIS workflow automation
- document-based geospatial analysis